"Loose coupling is a gift your future self will thank you for."

MODULE 22

Spring Core and Inversion of Control

Build loosely coupled, maintainable, and testable enterprise applications with Spring's IoC container and dependency injection.


TEKsystems
Global Services
WORKFORCE
DEVELOPMENT

WHY SPRING DOMINATES ENTERPRISE JAVA DEVELOPMENT

Spring provides everything enterprises need to build robust, scalable, secure, maintainable applications.

KEY REASONS

🚀 Productivity
Conventions and auto-config speed up delivery.

🏢 Enterprise Scale
Proven stability for high-traffic applications.

🌐 Full Ecosystem
Web, Data, Security, Messaging, Cloud, Testing.

🔒 Secure by Design
Deep integration with Spring Security.

☁️ Future-Ready
Supports microservices and cloud-native design.

🌍 Trusted Community
Huge community and wide enterprise adoption.

BUSINESS IMPACT

- **Faster development —** focus on business logic, not infrastructure.
- **Higher quality —** cleaner, maintainable, testable code.
- **Better scalability —** well-structured applications that grow with the business.

KEY TAKEAWAY Spring is not just a framework — it's a complete platform for production-ready, enterprise-grade applications.

MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to:

- **Understand Spring Fundamentals —** explain what Spring is, the problems it solves, and the ecosystem/architecture.
- **Explain Inversion of Control (IoC) —** describe how the IoC container manages object creation and dependencies.
- **Apply Dependency Injection —** implement and differentiate constructor, setter, and field injection.
- **Work with Spring Beans —** define beans, understand their lifecycle, and configure singleton/prototype scopes.
- **Use Component Scanning and Stereotypes —** apply @Component, @Service, @Repository, and @Controller.
- **Design Loosely Coupled Applications —** build maintainable, testable systems using IoC and bean relationships.

KEY TAKEAWAY These objectives build the core concepts and hands-on skills to build robust, maintainable enterprise applications.

ENTERPRISE SCENARIO: BUILDING MAINTAINABLE APPLICATIONS

A large-scale Order Management System must handle high traffic, evolve continuously, and integrate reliably.

BUSINESS CHALLENGES

- **Frequent Changes —** requirements shift constantly; code must adapt without breaking.
- **Tight Coupling —** makes testing, maintenance, and updates difficult and risky.
- **Complex Dependencies —** databases, external APIs, messaging, third-party libraries.
- **Large Teams —** multiple teams must work in parallel with minimal conflicts.

HOW SPRING CORE (IoC & DI) HELPS

- **Loose Coupling —** components depend on abstractions, not concrete implementations.
- **Dependency Injection —** the container manages dependencies and their lifecycle.
- **Easier Testing —** mocks and stubs are injected for unit and integration tests.
- **Scalability —** supports modular, extensible, enterprise-grade applications.

BUSINESS OUTCOMES

Faster delivery | **Fewer defects** | **Easier onboarding** | **Lower maintenance cost** | **Better resilience**

KEY TAKEAWAY Spring IoC and DI turn a tangled dependency graph into a maintainable, testable, and scalable application.

WHAT IS THE SPRING FRAMEWORK?

A comprehensive, lightweight, open-source framework for building enterprise-grade Java applications.

Spring provides a programming and configuration model that helps developers build robust, maintainable, testable applications quickly.

KEY CAPABILITIES

Core Container

Provides the IoC container and dependency injection.

AOP Support

Aspect-oriented programming for cross-cutting concerns.

Rich Ecosystem

Data access, web, security, messaging, and testing.

Enterprise Ready

Built for reliability, scalability, and performance.

WHY SPRING WAS CREATED

- Simplify complex enterprise application development.
- Promote loose coupling through Inversion of Control (IoC).
- Improve testability, maintainability, and reusability of code.
- Accelerate development and increase productivity.

KEY TAKEAWAY Spring provides the foundation for modern Java applications so you can focus on business logic, not boilerplate.

PROBLEMS SOLVED BY SPRING

Spring addresses the core complexities of enterprise Java development with proven solutions.

Common Problem	How Spring Solves It
Tight Coupling	IoC & DI — Spring manages object creation and wiring, promoting loose coupling.
Complex Object Management	IoC Container — Spring creates, configures, and manages objects automatically.
Difficult Maintenance	Centralized Configuration — externalized, annotation-based configuration improves clarity.
Poor Testability	Testable Code — loose coupling makes components easy to mock and test in isolation.
Repetitive Boilerplate	Cross-Cutting Concerns (AOP) — declarative transactions, security, caching, logging.

KEY TAKEAWAY Spring eliminates complexity and boilerplate, helping teams build robust, maintainable, scalable applications.

SPRING ECOSYSTEM OVERVIEW

A family of projects that work seamlessly together to build modern, enterprise-ready Java applications.

Core Foundation

Spring Framework, Context, SpEL — building blocks every other project uses.

Web

Spring MVC, WebFlux, Boot, Thymeleaf for web apps and REST services.

Data Access

Spring Data JPA, JDBC, MongoDB, Redis for SQL and NoSQL.

Security

Spring Security and OAuth2 for authentication and authorization.

Integration & Messaging

Spring Integration, Kafka, AMQP, JMS for connecting systems.

CLOUD & DEVOPS ENABLEMENT

Spring Boot | Spring Cloud | Spring Cloud Config | Spring Cloud Gateway

BENEFITS OF THE ECOSYSTEM

- Faster development of production-ready applications.
- High productivity — focus on business logic, not infrastructure.
- Large, active community and rich documentation.

KEY TAKEAWAY Spring is more than a framework — it's a complete ecosystem that accelerates enterprise Java development.

SPRING ARCHITECTURE OVERVIEW

A layered architecture where each layer builds on the one below it, with the IoC container at the core.

Layer	Key Components	Responsibility
Application	Web UI, REST Controllers, Mobile Clients	Handles requests/responses; exposes REST APIs.
Service	@Service beans, business logic	Business logic and workflows; uses the data layer.
Data Access	@Repository beans, Spring Data JPA	Abstracts persistence; CRUD operations and queries.
Infrastructure	DataSource, Transactions, Security, Messaging	Technical capabilities shared across the app.
Core Container	IoC Container — Beans, DI, Lifecycle, Scopes	Creates, injects, configures, and manages every bean.

CROSS-CUTTING CONCERNS (APPLIED ACROSS ALL LAYERS)

Logging **Validation** **AOP** **Monitoring** **Exception Handling**

KEY TAKEAWAY Spring's layered architecture promotes maintainability, scalability, and testability through IoC and DI.

UNDERSTANDING TIGHT COUPLING

Tight coupling occurs when a class depends directly on another class's concrete implementation.

OrderService.java (tightly coupled)

```
public class OrderService {
    // Direct dependency — hard-coded
    private EmailService emailService
        = new EmailService();

    public void placeOrder(Order order) {
        emailService.sendOrderConfirmation(order);
    }
}
```

WHAT IS TIGHT COUPLING?

- One class knows too much about another class's internals.
- It depends on a concrete implementation, not an abstraction.
- Any change to the dependency forces a change to the client.
- Harder to test, extend, and reuse in isolation.

Real-world analogy: a car welded to a specific engine — swapping the engine means rebuilding the entire car.

KEY TAKEAWAY Tight coupling makes code hard to change, test, and maintain — the opposite of what enterprise systems need.

PROBLEMS WITH TIGHT COUPLING

As applications grow, tight coupling creates strong dependencies that compound into real engineering costs.

Hard to Maintain

Changes ripple across multiple dependent classes.

Hard to Test

Isolated unit testing needs complex mocks or setups.

Low Flexibility

Difficult to replace or extend functionality.

Poor Reusability

Tightly coupled classes can't be reused elsewhere.

Ripple Effect

A small change can break many unrelated classes.

Slower Development

More time spent tracing dependencies than building.

Difficult to Scale

Dependency management gets complex and error-prone.

Higher Cost

More bugs, more testing, more maintenance overall.

KEY TAKEAWAY Loosely coupled systems are easier to change, test, extend, and scale than tightly coupled ones.

WHAT IS INVERSION OF CONTROL?

IoC transfers control of object creation and wiring to a container — the framework calls your code, not the reverse.

Without IoC

```
public class OrderService {
    private EmailService emailService;

    public OrderService() {
        emailService = new EmailService();
    }
}
// OrderService creates its own dependency
```

With IoC (Spring Container)

```
@Service
public class OrderService {
    private final EmailService emailService;

    @Autowired
    public OrderService(
            EmailService emailService) {
        this.emailService = emailService;
    }
}
```

THE IoC CONTAINER IS RESPONSIBLE FOR:

- **Creating objects**
- **Wiring dependencies**
- **Managing lifecycle**
- **Providing configuration**

KEY TAKEAWAY IoC lets you focus on business logic while the framework takes care of the plumbing.

BENEFITS OF IoC (INVERSION OF CONTROL)

IoC, managed by the Spring container, leads to better-designed, easier-to-maintain, enterprise-ready applications.

Loose Coupling

Objects depend on abstractions, not concrete classes.

Better Maintainability

Well-separated responsibilities are easier to modify.

Improved Testability

Dependencies can be mocked or stubbed for unit tests.

Greater Flexibility

Swap implementations without changing client code.

Scalability

Modular design eases growth and new integrations.

Enterprise Ready

Foundation for AOP, transactions, security, and more.

KEY TAKEAWAY IoC promotes loose coupling, reusability, and maintainability — the building blocks of scalable applications.

IoC CONTAINER OVERVIEW (1 OF 2)

Phase 1 — the container reads metadata, then creates and wires the bean objects it describes.

1. **Read Metadata** — XML or annotations
2. **Create Beans** — Instantiate bean objects
3. **Wire Dependencies** — Inject required beans

WHAT HAPPENS IN THIS PHASE

- **Read Metadata —** the container reads bean definitions from XML or annotations.
- **Create Beans —** it instantiates each described bean object.
- **Wire Dependencies —** it injects the required collaborators into each bean.

KEY TAKEAWAY Before a bean can be used, the container must know about it, create it, and wire its dependencies.

IoC CONTAINER OVERVIEW (2 OF 2)

Phase 2 — the container configures each bean, makes it available, and later manages its lifecycle.

1. **Configure Beans**
   Apply properties & settings
2. **Ready to Use**
   Fully initialized beans
3. **Manage Lifecycle**
   Cleanup and destruction

Container Type	Characteristics
BeanFactory	Basic container; lazy initialization; used in resource-constrained environments.
ApplicationContext	Enterprise-ready superset; eager initialization; supports AOP, events, i18n, validation.

KEY TAKEAWAY The IoC Container is Spring's backbone — it promotes loose coupling, modularity, and testability.

WHAT IS DEPENDENCY INJECTION?

DI is a design pattern where an object receives its dependencies from the container instead of creating them.

Constructor Injection

Provided via the constructor. Recommended — ensures immutability and testability.

Setter Injection

Provided via setter methods. Good for optional, changeable dependencies.

Field Injection

Injected directly into fields via @Autowired. Easy but hard to test — avoid in production.

BENEFITS OF DEPENDENCY INJECTION

Reduces Coupling | **Improves Testability** | **Increases Flexibility** | **Enhances Maintainability**

HOW IT WORKS

- The IoC Container creates and configures the dependency.
- The container injects the dependency into the target object.
- The dependent object is ready to use.

KEY TAKEAWAY Dependency Injection lets objects stay loosely coupled while the IoC container handles the wiring.

CONSTRUCTOR INJECTION

The preferred way to inject dependencies in Spring — required dependencies provided through the constructor.

WHY CONSTRUCTOR INJECTION?

- **Mandatory Dependencies —** ensures required dependencies are provided at creation time.
- **Immutability —** the dependency field can be declared final.
- **Easy to Test —** dependencies are explicit; no reflection needed for unit tests.
- **No Partial State —** the object is fully initialized when constructed.
- **Best Practice —** recommended by the Spring team as the default choice.

HOW IT WORKS

1. **Creates Dependency**
   EmailService bean created
2. **Injects via Constructor**
   Passed into OrderService()
3. **Bean Ready to Use**
   OrderService fully wired

KEY TAKEAWAY Constructor injection is Spring's recommended default — it produces robust, testable, maintainable code.

CONSTRUCTOR INJECTION — EXAMPLE

A complete constructor-injected service, plus the equivalent Java configuration.

OrderService.java

```
@Service
public class OrderService {
    private final EmailService emailService;

    // Constructor Injection
    public OrderService(EmailService emailService) {
        this.emailService = emailService;
    }

    public void placeOrder() {
        emailService.sendEmail("Order placed");
    }
}
```

AppConfig.java (Java Config)

```
@Configuration
public class AppConfig {
    @Bean
    public OrderService orderService(
            EmailService emailService) {
        return new OrderService(emailService);
    }
}
```

KEY POINTS

- Dependencies pass through the constructor and are final.
- The class cannot be built without its dependencies.
- Since Spring 4.3+, a single constructor needs no @Autowired.

KEY TAKEAWAY If a class has only one constructor, Spring wires it automatically — no @Autowired required.

SETTER INJECTION

Dependencies are provided through setter methods after construction — useful for optional dependencies.

WHY SETTER INJECTION?

- **Optional Dependencies —** useful when a dependency is not mandatory.
- **Flexibility —** dependencies can be changed or updated after creation.
- **Reusability —** the same object can be reused with different configurations.
- **Easier Large Configuration —** helpful in XML configs with many properties.
- **Legacy Integration —** works with code that expects a no-arg constructor.

HOW IT WORKS

1. **Container Creates Bean**
   No-arg constructor used
2. **Calls Setter Method**
   setEmailService() invoked
3. **Bean Ready to Use**
   Dependency now attached

KEY TAKEAWAY Setter injection offers flexibility for optional dependencies, but constructor injection stays preferred.

SETTER INJECTION — EXAMPLE

Dependency provided through a setter, plus the equivalent XML configuration.

OrderService.java

```java
public class OrderService {
    private EmailService emailService;

    // Setter Injection
    @Autowired
    public void setEmailService(
            EmailService emailService) {
        this.emailService = emailService;
    }
}
```

XML Configuration

```xml
<bean id="orderService"
      class="com.example.OrderService">
    <property name="emailService"
              ref="emailService" />
</bean>
```

BEST USE CASES

- Optional deps
- Configurable props
- Runtime-changeable
- Legacy code

KEY TAKEAWAY Use setter injection for optional or reconfigurable dependencies — not as the default choice.

FIELD INJECTION

Dependencies are injected directly into fields via reflection — the least recommended DI style in Spring.

ADVANTAGES

- Less boilerplate — no constructor or setter methods.
- Quick to implement for small demos or prototypes.
- Simple, compact class definitions.

DISADVANTAGES

- Not test-friendly — objects can't be built outside the container.
- Hidden dependencies — not visible in the class definition.
- Breaks encapsulation by injecting into private fields directly.

The Spring team recommends constructor injection over field injection for production code.

KEY TAKEAWAY Field injection works, but it hides dependencies and makes testing harder — prefer constructor injection.

FIELD INJECTION — EXAMPLE

Dependency injected directly into a private field via @Autowired.

OrderService.java

```
public class OrderService {

    @Autowired
    private EmailService emailService;

    public void placeOrder() {
        emailService.sendEmail("Order placed");
    }
}
```

KEY POINTS

- @Autowired is applied directly to the field.
- The container injects the dependency via reflection.
- Cannot be unit tested without a Spring context.
- Should be avoided in production code.

BEST USE CASES (LIMITED)

- Quick prototypes
- Legacy applications
- Minimal demo code

KEY TAKEAWAY Use constructor injection for mandatory deps and setter injection for optional ones — field injection is a last resort.

INJECTION BEST PRACTICES

Practical guidelines for using Spring dependency injection effectively in enterprise code.

#	Practice	Why It Matters
1	Prefer Constructor Injection	Default choice — required, immutable, testable dependencies.
2	Use Setter Injection for Optional Deps	Only when a dependency may be absent or change later.
3	Avoid Field Injection	Hides dependencies and blocks testing outside Spring.
4	Program to Interfaces	Depend on abstractions; let Spring supply the implementation.
5	Keep Beans Focused	Single responsibility — avoid god classes.
6	Use @Qualifier When Needed	Disambiguate when multiple beans share a type.
7	Avoid Circular Dependencies	Redesign rather than field-inject around a cycle.
8	Test With Mocked Dependencies	Verify business logic in isolation from Spring.

KEY TAKEAWAY Use constructor injection by default, keep dependencies explicit, and test business logic in isolation.

WHAT IS A SPRING BEAN?

Any object created, configured, wired, and managed by the Spring IoC container.

Container-Managed

The IoC container creates and manages the bean.

Configured as Metadata

Defined via XML, Java config, or annotations.

Dependencies Injected

The container injects required collaborators.

Lifecycle Managed

Container controls init, use, and destroy phases.

Reusable & Scalable

Promotes reuse, loose coupling, easy testing.

TYPES OF SPRING BEANS (BY SCOPE)

Scope	Description
Singleton (default)	One shared instance per Spring container.
Prototype	A new instance every time the bean is requested.
Request / Session	Web-aware scopes — one instance per HTTP request or session.

KEY TAKEAWAY A Spring Bean is an object the container creates, wires, and manages — promoting loose coupling and reuse.

BEAN LIFECYCLE OVERVIEW (1 OF 2)

Phase 1 — the container brings a bean into existence and prepares it for initialization.

1	2	3	4
Instantiate	**Populate Properties**	**Aware Callbacks**	**Before-Init**
Constructor runs	Dependencies injected	BeanNameAware, etc.	BeanPostProcessor

WHAT HAPPENS IN THIS PHASE

- **Instantiate —** the container calls the bean's constructor to create the raw object.
- **Populate Properties —** constructor or setter injection wires the bean's dependencies.
- **Aware Callbacks —** BeanNameAware, BeanFactoryAware, and similar callbacks run if implemented.
- **Before-Init —** any registered BeanPostProcessor.postProcessBeforeInitialization() runs.

KEY TAKEAWAY Before a bean is ever used, the container must create it, wire it, and run any pre-init callbacks.

BEAN LIFECYCLE OVERVIEW (2 OF 2)

Phase 2 — the bean is initialized, used by the application, and eventually destroyed.

1. **Initialize**
   afterPropertiesSet() / init
2. **Ready to Use**
   Bean available in context
3. **Destroy**
   @PreDestroy / destroy

Lifecycle Callbacks Example

```
@Component
public class OrderService {
    @PostConstruct
    void init() {
        // setup logic — runs once after dependencies are injected
    }
    @PreDestroy
    void shutdown() {
        // cleanup logic — runs before the bean is destroyed
    }
}
```

KEY TAKEAWAY Use @PostConstruct for setup and @PreDestroy for cleanup — proper lifecycle management prevents resource leaks.

BEAN SCOPES IN SPRING

A bean's scope determines how many instances the container creates and how long each one lives.

Scope	Instances	Typical Use Case
singleton (default)	1 per container	Stateless services, repositories, configuration beans.
prototype	New instance per request	Stateful, non-shared, or task-specific objects.
request (web-aware)	1 per HTTP request	Web controllers, request-scoped data.
session (web-aware)	1 per HTTP session	User session data, shopping cart.

DECLARING SCOPE

@Scope("singleton") **@Scope("prototype")** **@Scope(value="request", proxyMode=...)**

request, session, application, and websocket scopes are web-aware and only work in a web-aware Spring application.

KEY TAKEAWAY Choose singleton for shared, stateless beans and prototype for stateful or non-shared beans.

SINGLETON SCOPE

The default Spring scope — only one instance is created per container and shared across the application.

Single Instance

Only one instance exists for the entire container.

Shared Everywhere

The same instance is injected into every dependent bean.

Created at Startup

Instantiated eagerly when the container starts, by default.

Best for Stateless Beans

Ideal for services, repositories, and utilities.

Example — Singleton by Default

```
@Service
public class OrderService {
    public void placeOrder() { /* ... */ }
}
// No @Scope needed — singleton is the default
```

- Stored in the container's singleton cache.
- All clients receive the same instance.
- Thread-safe only if internal state is handled carefully.

KEY TAKEAWAY Use singleton scope for stateless, thread-safe beans shared across the application — Spring's default scope.

PROTOTYPE SCOPE

A new instance is created every time the bean is requested — the container does not cache it.

New Instance Every Time

A fresh instance is created on every request.

Not Cached

The container does not store or reuse the instance.

Independent State

Each instance has its own, isolated state.

No Auto-Destroy

Container skips destroy callbacks; client manages cleanup.

Example — @Scope("prototype")

```
@Component
@Scope("prototype")
public class ReportGenerator {
    private final UUID id = UUID.randomUUID();
}
// Each request gets a different instance/id
```

- Use for stateful beans or per-request/user data.
- Retrieve via context.getBean(...) each time.
- p1 != p2 for two separate lookups.

KEY TAKEAWAY Use prototype scope when each client needs its own fresh, stateful instance.

COMPONENT SCANNING OVERVIEW

Spring automatically detects and registers annotated classes as beans, eliminating manual bean declarations.

1. **Base Package Specified** — Point Spring at a root package
2. **Classpath Scan** — Sub-packages scanned too
3. **Component Detection** — Stereotype classes found
4. **Bean Registration** — Registered in the context
5. **Ready for Injection** — Beans available for DI

Enabling Component Scanning

```
@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(
            DemoApplication.class, args);
    }
}
```

STEREOTYPES DETECTED

- **@Component —** generic Spring-managed component.
- **@Service —** business/service layer.
- **@Repository —** data access layer (DAO).
- **@Controller —** web/MVC layer.

KEY TAKEAWAY Component scanning reduces manual configuration and enables a clean, modular application architecture.

@COMPONENT ANNOTATION

The generic stereotype — marks a class as a Spring-managed bean eligible for component scanning.

EmailService.java

```
@Component
public class EmailService {
    public void sendEmail(String to, String body) {
        // send logic
    }
}
```

Annotation	Purpose
@Component	Generic stereotype for any managed component.
@Service	Service layer (business logic).
@Repository	Data access layer (DAO).
@Controller	Web layer (Spring MVC).

BEST PRACTICES

- Use @Component only when no specialized stereotype fits; prefer @Service, @Repository, or @Controller for clarity.
- The registered bean name defaults to the class name in camelCase (e.g. emailService).

KEY TAKEAWAY @Component is the foundation of Spring's stereotypes — @Service, @Repository, and @Controller build on it.

@SERVICE ANNOTATION

Marks a class as a service-layer component that contains business logic.

UserService.java

```
@Service
public class UserService {
    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public User findById(Long id) {
        return userRepository.findById(id)
            .orElseThrow();
    }
}
```

BEST PRACTICES

- Keep business logic in @Service classes.
- Make services stateless whenever possible.
- Use constructor injection for dependencies.
- Follow the single responsibility principle.

KEY TAKEAWAY @Service marks the business logic layer, improving readability and enabling clean, layered architecture.

@REPOSITORY ANNOTATION

Marks a class as a data-access-layer (DAO) component with automatic exception translation.

UserRepository.java

```
@Repository
public class UserRepository {
    private final JdbcTemplate jdbcTemplate;

    public UserRepository(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    public User findById(Long id) {
        String sql = "SELECT * FROM users WHERE id = ?";
        return jdbcTemplate.queryForObject(sql,
            new Object[]{id}, new UserRowMapper());
    }
}
```

BEST PRACTICES

- Keep only data-access logic in @Repository classes.
- Let Spring translate SQL exceptions automatically.
- Prefer Spring Data JPA repositories when possible.
- Use meaningful names (UserRepository, OrderDao).

KEY TAKEAWAY @Repository enables automatic detection and exception translation for the persistence layer.

@CONTROLLER ANNOTATION

Marks a class as a web-layer component that handles HTTP requests and returns responses.

UserController.java

```
@Controller
@RequestMapping("/user")
public class UserController {
    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping("/{id}")
    public String getUser(@PathVariable Long id, Model model) {
        model.addAttribute("user", userService.findById(id));
        return "user-details";
    }
}
```

BEST PRACTICES

- Keep controllers thin — business logic belongs in @Service.
- Use meaningful request mappings and method names.
- Validate input and handle exceptions properly.
- Follow RESTful principles for API controllers.

KEY TAKEAWAY @Controller works with Spring MVC to route HTTP requests and delegate to the service layer.

BUILDING LOOSELY COUPLED SYSTEMS

Depend on abstractions, not concrete implementations — Spring's IoC container and DI make it practical.

TIGHTLY COUPLED

- OrderService depends on the concrete EmailService class.
- Difficult to change or replace EmailService.
- Harder to test — the real EmailService is required.

LOOSELY COUPLED

- OrderService depends on a PaymentService interface.
- Any implementation can be injected at runtime.
- Easier to test, maintain, and extend.

Programming to an Interface

```
public interface PaymentService {
    void pay(double amount);
}

@Service
public class CreditCardPaymentService implements PaymentService {
    public void pay(double amount) { /* charge card */ }
}
```

KEY TAKEAWAY Program to interfaces and let Spring inject the implementation — the key habit behind loose coupling.

SPRING BEAN RELATIONSHIPS

Beans collaborate through Dependency Injection — the container manages these relationships for you.

Relationship	Description
Dependency	One bean requires another to do its work (constructor/setter injection).
Association	Beans are loosely linked and can work independently.
Composition	Strong ownership — the owned bean cannot exist without the owner.
Inheritance	A child bean inherits configuration from a parent bean definition.
References	A bean holds a reference to a collection of other beans.

Dependency via Constructor Injection

```
@Service
public class OrderService {
    private final PaymentService paymentService;
    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

KEY TAKEAWAY Dependency Injection and the IoC container manage bean relationships, producing testable applications.

ENTERPRISE APPLICATION STRUCTURE

A clean, layered package structure keeps enterprise Spring applications maintainable and easy to navigate.

1. **Presentation**
2. **Business**
3. **Data Access**
4. **Database**
5. **External Systems**

Package Structure Example

```
com.example.bank
|-- controller      REST Controllers
|-- service         Business Services
|-- repository      Data Access Interfaces
|-- entity          JPA Entities
|-- dto             Request / Response DTOs
`-- config          Configuration classes
```

BEST PRACTICES

- Keep controllers thin; delegate to services.
- Use interfaces for services and repositories.
- Leverage DI for loose coupling between layers.
- Handle exceptions consistently across layers.

KEY TAKEAWAY A clear layered structure with IoC-based wiring produces flexible, testable, future-ready systems.

LAB OVERVIEW — SPRING IoC AND DEPENDENCY INJECTION

Lab 22: Spring IoC and Dependency Injection — Northstar CRM Bean Graph

BUSINESS SCENARIO

- The Northstar CRM stores customer identity, contact details, lifecycle status, and accounts.
- Leadership freeze: Spring owns the object graph; constructor injection is preferred; every component carries a stereotype; the dependency graph is documented.
- Manual new InMemoryCustomerRepository() calls inside services block swapping persistence, metrics, and notifiers for tests and production.

LEARNING OBJECTIVES

- Explain IoC vs. dependency lookup — CRM services should never call new on collaborators.
- Declare @Component, @Service, and @Repository beans for the customer domain.
- Prefer constructor injection for CustomerService's dependencies.
- Observe bean lifecycle via @PostConstruct / @PreDestroy.
- Document and export a dependency graph for review.

WHAT YOU BUILD

- Copy lab21-crm to lab22-crm; declare @Repository/@Service beans; refactor CustomerService to constructor DI; wire @RestController; add @PostConstruct/@PreDestroy; prove unit + @SpringBootTest.

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan) and CUS-1002 (Ravi Singh) with correlation lab-request-001 anchor every example.

LAB TASKS AND DELIVERABLES

Northstar CRM Bean Graph — implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Branch Lab 21 and confirm the Spring Boot scaffold starts cleanly.
2	Model the Customer domain type without Spring annotations.
3	Declare @Repository / @Service beans for repository and notifier collaborators.
4	Refactor CustomerService to constructor injection with final fields.
5	Wire CustomerController as a Spring MVC bean via constructor injection.
6	Add @PostConstruct / @PreDestroy lifecycle logging to CustomerService.
7	Prove testability — pure unit test plus a @SpringBootTest integration test.
8	Document docs/dependency-graph.md and run the failure experiments.

EXPECTED DELIVERABLES

- CustomerService, CustomerRepository, NotificationService as constructor-injected beans.
- Lifecycle evidence (@PostConstruct/@PreDestroy).
- Unit test with fakes + a @SpringBootTest IT.
- docs/dependency-graph.md matching the real constructors.
- Evidence for CUS-1001 / CUS-1002 with correlation lab-request-001.

RUBRIC (100 MARKS)

- Core implementation 30 · Integration 15 · Failure handling 15 · Tests 10 · Security 10 · Docs 10 · Env 10

KEY TAKEAWAY Field @Autowired as the primary pattern, or any new of a Spring-managed collaborator, is an automatic deduction.

MODULE 22 SUMMARY

Spring Core and Inversion of Control — what we covered and how it fits together.

KEY CONCEPTS COVERED

IoC Container
Creates, configures, and manages beans and their lifecycle.

Dependency Injection
Constructor, setter, and field injection wire collaborators.

Bean Lifecycle & Scope
Init/destroy callbacks; singleton and prototype scopes.

Configuration Options
XML, Java config, or annotations define beans.

Loose Coupling
Program to interfaces; let Spring supply the implementation.

Stereotypes
@Component, @Service, @Repository, @Controller drive scanning.

MODULE FLOW RECAP

1. **IoC Container**
2. **Dependency Injection**
3. **Bean Lifecycle & Scope**
4. **Component Scanning**
5. **Loosely Coupled Design**

KEY TAKEAWAY Spring IoC and Dependency Injection enable loose coupling and better design — the foundation for everything ahead.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of Spring IoC and dependency injection.

1. What does the Spring IoC container primarily manage?

A. Database connections

B. Bean creation, configuration, and lifecycle

C. HTTP sessions only

D. File I/O operations

2. Which injection type does the Spring team recommend for mandatory dependencies?

A. Field injection

B. Setter injection

C. Constructor injection

D. Static injection

3. What is the default scope of a Spring bean?

A. Prototype

B. Request

C. Session

D. Singleton

4. Which annotation marks a data-access-layer (DAO) component?

A. @Controller

B. @Service

C. @Repository

D. @Configuration

KEY TAKEAWAY IoC and DI are the foundation of every Spring-managed enterprise application.

KNOWLEDGE CHECK (2 OF 2)

Four more questions on beans, scopes, and component scanning.

5. Why is field injection discouraged in production code?

A. It is slower at runtime
B. It hides dependencies and is hard to test without Spring
C. It cannot use interfaces
D. It requires XML configuration

6. Which lifecycle annotation runs cleanup logic before a bean is destroyed?

A. @PostConstruct
B. @PreDestroy
C. @Bean
D. @Scope

7. A new instance is created every time a bean is requested under which scope?

A. Singleton
B. Prototype
C. Application
D. Request

8. What lets Spring automatically detect @Service and @Repository classes?

A. Manual XML bean registration
B. Component scanning
C. The JVM classloader alone
D. Reflection API alone

KEY TAKEAWAY Component scanning plus stereotype annotations let Spring find and wire your beans with almost no manual config.

MODULE 22 COMPLETE

Spring Core and Inversion of Control

You can now build Spring beans, apply constructor/setter/field injection, manage bean lifecycle and scope, and design loosely coupled enterprise applications.